table maintaining *ceil(log2^bN)* rows with *2b -1* entries in each row, where *b* is a configurable parameter that we have used as 4 in our case in order to consider the key as a number in the hexadecimal system.

Hence, we will be doing our routing operation in the hexadecimal system on the 128-bit value key, which we get as 32 hexadecimal characters of the md5 digest and IPv4 plus port for *nodeId*. These *2b -1* entries at the n^{th} row in the routing table refer to a node within the application's network, whose *nodeId* shares an n-length prefix with the current node's *nodeId* and its $(n+1)^{th}$ digit (hexadecimal character) is different from the present node's *nodeId*. The idea is a bit similar to the way the network id for an IP is decided while routing in a network. Here, it uses bitwise AND of IP with a subnet mask, and goes for the node with a longer run of 1's in case of a clash.

Similarly, choose the routing mechanism for a message associated with a key based upon its key value. The neighbourhood set is intended to store IP addresses and *nodeIds* for M closest nodes to the present node according to proximity. The leaf set L is a set of nodes with L/2 nodes having *nodeId* smaller than the present node's *nodeId* in the *nodeId* space, and are the closest such nodes in this space. Similarly, there are L/2 nodes with *nodeIds* greater than the present node's *nodeId*.

Routing of a message for the destination node happens as per the algorithm described below.

Routing algorithm (route to destination node)

- Initially, we check if the desired node lies within the range of leaf sets.
  - If yes, then in a single step we can reach the destination.
- Say, 'n' is the common prefix length shared by the key and current node. And 'j' is the n^{th} value in messages' key.
- Check for a not NULL entry in the n^{th} row and j^{th} column.
  - Route to the existing entry if not NULL.
- Take the union of all tables (R U M U L) and route to the most appropriate node.

To do a repair of the node's state, the following algorithm is used.

Repair algorithm

- Leaf Set Repair
  - For the smaller *nodeIds* that lie in the left leaf set, request the leftmost leaf node for its leaf sets. Search for an entry with *nodeId* less than the leftmost leaf in the request leaf set. If found, insert it into the left leaf set.
  - If such an entry cannot be found, then repeat the same procedure for the second leftmost leaf node and so on until an appropriate entry is found.
  - Repeat the same procedure for the right leaf set, except that the rightmost leaf node will be requested first, and then move one step left if the relevant entry is not found.
- Neighbourhood Set Repair
  - Request the neighbourhood set from the rearmost neighbour of the present node. If a new entry is found, then insert it into the neighbourhood set. Otherwise, head forward repeating the same process with the node next to the previously selected node.
- Routing Table Update
  - To repair *R[n][j]*, request for the *R[n][j]* entries from nodes *R[p][j]* for all *j!=i*.
  - When the desired entries are not found, then it goes on following the same procedure with the next row.

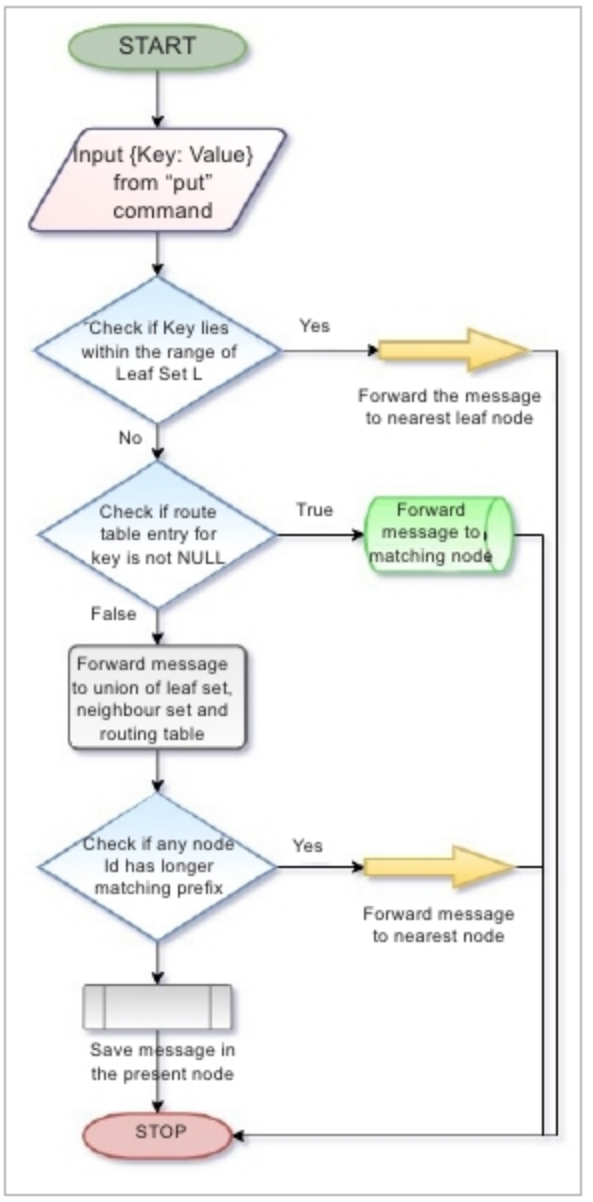

Figure 4: Stages of routing procedure for message with an associated key

In Figure 4, a flow diagram demonstrates the working of the routing algorithm for values based on their keys involving three crucial stages of decision making.

When a new node X asks to join the application's network to an existing node A, a special message 'join' with X's *nodeId* is routed to the numerically closest live nodes like any other message. All nodes encountered during routing send their state information to node X as a response to the join request. In case of a node departure, the failed node within the leaf set is replaced. Every stored data is replicated across three different nodes.

Upon joining, a new node is provided with the data relevant to it, while routing. Routing table entries are fixed when an unavailable node is detected while routing. This is commented as 'Lazy repair' in the code. Every 30 seconds a timely leaf set repair is done. When a node quits the network, a replica of its data is created before it exits the network. This is done